Построение пайплайнов данных

Физический слой: файловые системы и форматы данных

Дмитрий Малахов

Максим Новосельцев

6 февраля 2026 года

Распределённые файловые системы

Почему распределённые?

- Объём > возможности одного сервера
- Требуется отказоустойчивость
- Горизонтальное масштабирование

Основные системы:

HDFS: NameNode + DataNodes

Объектные хранилища (Simple Storage Service, Google Cloud Storage)

Общие принципы: шардирование, локализация

Apache Ozone

- Объединяет модели HDFS и S3 (O3FS, S3 API)
- Поддерживает локальные вычисления в отличии от классических S3
- В большинстве случаев работает быстрее HDFS
- Имеет больше возможностей к масштабированию, чем HDFS
- Один кластер для аналитики, сырых данных и облачных приложений
- Активно развивается

Критерии выбора формата данных

Что оцениваем:

1. Объём хранения
2. Скорость чтения/записи
3. Поддержка схемы и возможности ее эволюции
4. Совместимость с инструментами
5. Возможность разделения (splittability)

CSV/TSV

Особенности :

- Человекочитаемый
- Универсальный (условно)
- Простой

Проблемы :

- Нет схемы, типов
- Плохое сжатие (нет по умолчанию)
- Не splittable (условно)

Use case: маленькие объёмы и нужна интерпретируемость

JSON

Особенности:

- Человекочитаемый (условно)
- Полуструктурированный
- Вложенные структуры
- Широкая поддержка в вебе

Проблемы:

- Нет схемы (типы ограничены)
- Высокая избыточность
- Плохое сжатие (нет по умолчанию)
- Не splittable (условно)

Use case: API, HTTP-логи, legacy, как у CSV/TSV (сложнее модель)

Avro

Особенности:

- Схема в заголовке/registry
- Эволюция схемы
- RPC поддержка
- Поддержка сжатия
- Splittable
- В остальном схож с Json

Проблемы:

- Хранит данные построчно
- Типизация сильно ограничена

Use case: API, очереди сообщений, логи, аналитика

Protobuf

Особенности:

- Бинарный, быстрый, компактный
- Строгая схема и кодогенерация (.proto)
- Эволюция схемы
- gRPC интеграция
- Широкая поддержка

Проблемы:

- Нет сжатия
- Не splittable (условно)

Use case: API, очереди сообщений

Parquet

Особенности:

- Колоночное хранение
- Эффективное сжатие
- Predicate pushdown
- Широкая поддержка

Проблемы:

- Только для табличных данных (условно)
- В ряде случаев может быть менее эффективным, чем Avro

Use case: аналитические запросы, агрегации

Parquet + Protobuf гибрид

Проблема: Parquet неэффективен для сложных структур

Решение: Protobuf в колонке Parquet

Таблица:

- date (DATE) → фильтрация
- user_id (INT) → джойны
- metrics (BINARY) → Protobuf блоб

Parquet + Binary гибрид

Преимущества:

- Гибкость и эффективность бинарного формата (например Protobuf)
- Аналитика Parquet
- Двойное сжатие

Главная проблема - сложность

Use case: там где не работает ничего другого

Sequence File

Особенности:

- Привязан к Hadoop экосистеме
- Позволяет хранить любые пары ключ-значение
- Splittable

Проблемы:

- Часто менее эффективен, чем Parquet
- Ограниченная оптимизация

Use case: неструктурированные записи

ORC

Особенности:

- Колоночное хранение
- Индексы, min/max
- ACID в Hive

Проблемы:

- Часто менее эффективнее, чем Parquet
- Вероятно будет постепенно отмирать

Use case: оптимизация под Hive экосистему, аналитику

Сравнительная таблица

Формат	Тип	Схема	Чтение (аналитика)	Запись	Использование
CSV	Row	Нет	Медленно	Быстро	Обмен
JSON	Row	Нет	Медленно	Быстро	Веб/API
Parquet	Column	Есть	Очень быстро	Медленно	Аналитика
Avro	Row	Есть+evolution	Средне	Быстро	Стриминг
Protobuf	Row	Строгая	Медленно	Очень быстро	Микросервисы

Паттерны доступа

1. Аналитика (OLAP):

Много чтения, агрегации → Parquet

2. Потоковая обработка:

Частые записи, evolution → Avro

3. Микросервисы:

RPC, межсервисная коммуникация → Protobuf

4. Гибрид:

Сложные структуры + аналитика → Parquet с Protobuf колонкой

Оптимизация

Сжатие:

- GZIP: хорошее сжатие, умеренная скорость
- Snappy: умеренное сжатие, высокая скорость
- ZSTD: баланс

Размер блоков:

- Больше блоков → лучше параллелизм
- Меньше блоков → меньше overhead

А как насчет СУБД?

- Не так хорошо подходят для параллельной обработки (даже NoSQL)
- Не рассчитаны на хранение плохо структурированных данных
- При этом транзакционность и в ряде случаев большая скорость работы требуют использования именно этого решения, что приводит к необходимости синхронизации двух архитектур и дублирования данных между ними
- Появление архитектуры Data Lakehouse призвано взять лучшее из двух подходов

Итоги:

- Для построения Data Lake используем **Apache Ozone**
- Модель данных нужно проектировать так, чтобы потребность в дополнительном DWH была минимальна (**строим Data Lakehouse**)
- **Формат** данных сильно **зависит** от их **природы** и того, как они будут **обрабатываться**

Домашнее задание

Тест производительности 3+ форматов

Задачи:

- Подготовить датасет ~10GB
- Протестировать минимум 3 формата

Измерить:

- Объём хранения
- Время записи
- Время чтения (full scan, выборка, агрегация)

Форматы: Parquet, Avro + на выбор (CSV, JSON, Protobuf)